

Polyhedral Specification and Code Generation of Sparse Tensor Contraction with Co-iteration

TUOWEN ZHAO, University of Utah, USA
TOBI POPOOLA, Boise State University, USA
MARY HALL, University of Utah, USA
CATHERINE OLSCHANOWSKY, Boise State University, USA
MICHELLE STROUT, University of Arizona, USA

This article presents a code generator for sparse tensor contraction computations. It leverages a mathematical representation of loop nest computations in the sparse polyhedral framework (SPF), which extends the polyhedral model to support non-affine computations, such as those that arise in sparse tensors. SPF is extended to perform layout specification, optimization, and code generation of sparse tensor code: (1) We develop a polyhedral layout specification that decouples iteration spaces for layout and computation; and (2) we develop efficient co-iteration of sparse tensors by combining polyhedra scanning over the layout of one sparse tensor with the synthesis of code to *find* corresponding elements in other tensors through an SMT solver.

We compare the generated code with that produced by a state-of-the-art tensor compiler, TACO. We achieve on average 1.63× faster parallel performance than TACO on sparse-sparse co-iteration and describe how to improve that to 2.72× average speedup by switching the find algorithms. We also demonstrate that decoupling iteration spaces of layout and computation enables additional layout and computation combinations to be supported.

CCS Concepts: • **Software and its engineering → Source code generation**; **Domain specific languages;**

Additional Key Words and Phrases: Data layout, sparse tensor contraction, polyhedral compilation, code synthesis, uninterpreted functions, index array properties

ACM Reference format:
Tuowen Zhao, Tobi Popoola, Mary Hall, Catherine Olschanowsky, and Michelle Strout. 2022. Polyhedral Specification and Code Generation of Sparse Tensor Contraction with Co-iteration. *ACM Trans. Arch. Code Optim.* 20, 1, Article 16 (December 2022), 26 pages.
https://doi.org/10.1145/3566054

This research was supported in part by the Exascale Computing Project (17-SC-20-SC), a joint project of the U.S. Department of Energy's Office of Science and National Nuclear Security Administration and by the National Science Foundation under project CCF-2107556.

Authors' addresses: T. Zhao and M. Hall, School of Computing, 50 S. Central Campus Drive, Salt Lake City, UT 84112; emails: ztuowen@gmail.com, mhall@cs.utah.edu; T. Popoola and C. Olschanowsky, Computer Science Department, 777 W. Main Street #364, Boise, ID 83702; emails: tobipopoola@u.boisestate.edu, catherineolschan@boisestate.edu; M. Strout, Department of Computer Science, P.O. Box 210077, Tucson, AZ 85721; email: mstrout@cs.arizona.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
1544-3566/2022/12-ART16 $15.00
https://doi.org/10.1145/3566054

1 INTRODUCTION

Tensor contractions are found in a wide variety of computations in data science, machine learning, and finite element methods [1, 8, 19, 26, 74]. Sparse tensors are tensors that contain a large number of zero values that have been compressed out to save memory and avoid unnecessary computation. A *layout* is used to represent a sparse tensor, which includes the nonzero values and a set of auxiliary data structures that relate nonzero values to their indices in the computation. Many sparse tensor layouts have been introduced to improve performance under different algorithmic contexts, sparsity patterns, and for different target architectures (for examples, see the survey by Langr and Tvrdik [46]).

A sparse tensor layout can be thought of as a *physical* description of the sparse tensor—how it is ordered in memory and the requisite auxiliary data structures that define its meaning. The *logical* view of the sparse tensor is its dense form, which is usually prohibitively large to represent in memory; the logical abstraction of the nonzeros must be preserved by the physical layout.

To optimize both computation and layout of sparse tensors, several sparse tensor compilers have been developed that generate optimized code from a dense description of the computation, using a sparse physical layout of the tensor [11, 41, 45]. Most recently, the **Tensor Algebra Compiler (TACO)** [41] uses *level formats* [20] to describe the physical storage of different index dimensions of a tensor, with each level also associated with an index dimension in the tensor computation. This layout description and the formulation using *merge lattices* allows dimensions from multiple sparse tensors to be *co-iterated*, which refers to matching coordinates of nonzeros in one sparse tensor to those in another sparse tensor. For example, in sparse dot product, if coordinate p is nonzero for one of the vectors, then the element-wise product is nonzero if and only if p is also nonzero in the other vector. TACO's support of co-iteration extends the applicability of tensor compilers.

We observe that the use of level formats and merge lattices couples the logical (computation's coordinate space) and physical (layout's position space) dimensions and their associated iteration ranges; consequently, this approach requires that each level in the layout must refer to a distinct index in the computation. Level formats are unable to directly support blocked layouts such as **block compressed sparse row (BCSR)**, which have additional physical dimensions not present in the computation. Additionally, generalizations of contraction that use the same loop index for multiple levels, e.g., computations along a matrix diagonal, cannot be directly supported due to conflicts in iteration ranges.

In this article, we separate the physical layout of the sparse tensor (layout's position space) from logical indices (computation's coordinate space) by preserving indices from both spaces and describing a mathematical relation between them. For this purpose, our representation extends the polyhedral model [13, 28–32, 56, 62], an abstraction used to represent integer sets and compose optimizations on loop nest computations, and compose computation with storage mappings [48, 55, 69, 72]. A rich set of affine code transformations can be described using the polyhedral framework and related mathematical representations including locality optimization [37, 57, 81, 82], automatic and semi-automatic parallelization [13, 18, 30, 37, 40], and auto-distribution [9, 39]. To support sparse computation involving non-affine loop bounds and indirect accesses in subscript expressions (e.g., A[B[i]]), our approach employs techniques from the **Sparse Polyhedral Framework (SPF)** [70, 71], which uses *uninterpreted functions* to represent values of auxiliary *index arrays* that are only known at runtime.

Prior work using SPF has not presented a solution to co-iteration of multiple sparse tensors [52, 68, 70, 77]. In this article, we extend SPF to support co-iteration by generating code that iterates over the layout of one sparse tensor's nonzeros and looks up corresponding nonzeros in other tensors with *find* operations.

This article makes the following contributions: (1) We describe a sparse tensor layout as a relation between the logical and physical space, which supports layouts that cannot be described when coordinate and position spaces are coupled; (2) We extend SPF to generate efficient co-iteration code through a combination of polyhedra scanning and code synthesis using a **satisfiability modulo theories (SMT)** solver. (3) We show how the use of SPF facilitates parallelization and composition of transformations, demonstrated by deriving data dependence relations and tiling the code; (4) We compare the proposed method with a state-of-the-art tensor algebra compiler, TACO. On sparse-sparse co-iteration in the sparse-matrix times sparse-vector experiments, we achieve on average 1.63× speedup against TACO using a find algorithm comparable to TACO's co-iteration implementation and improve that to 2.72× average speedup when we switch between find algorithms. We can even express cases when the input and output share sparsity structure such as in the mode-1 tensor-times-matrix computation, where we are able to achieve 4.27× average speedup on real-world 3D tensors.

2 BACKGROUND AND OVERVIEW

This section formulates tensor contraction code generation in the polyhedral framework for dense tensors and demonstrates how the sparse polyhedral framework represents the index arrays arising from sparse tensors. The end of the section gives an overview for the remainder of the article.

2.1 Tensor Contraction

Tensor contractions can be expressed using the *tensor index notation* described by Ricci and Levi-Civita [58], where dimensionality of input tensors is contracted along one or more dimensions. Examples from linear algebra include dot product, $y = A(i) * B(i)$, matrix-vector multiplication, $y(i) = A(i, j) * B(j)$, and matrix-matrix multiplication, $y(i, j) = A(i, k) * B(k, j)$. Higher-dimensional tensor contractions are common occurrences in machine learning and the finite element method. This notation expresses the accesses of the input and output tensors in the computation. Indices only appearing on the right-hand side, such as k in matrix-matrix multiplication, are commonly referred to as *summation* or *contraction* indices and introduce a data dependence; indices appearing on both sides, such as i and j, are commonly called the *external* or *free* indices. Because the contraction index k iterates over the second dimension of A and the first dimension of B at the same time, this behavior is referred to as co-iteration.

2.2 Polyhedral Framework

Polyhedral frameworks describe the instances of a statement's execution in a loop nest as a set of lattice points of polyhedra. Polyhedral compilers were designed to support computations that are in the *affine domain*, where loop bounds and subscript expressions are integer linear functions of loop indices and constants. Polyhedra are specified by a *Presburger formula* on index variables through affine constraints, logical operators, and existential operators. When specified this way, this set of lattice points are also called a *Presburger set*. Presburger sets and relations (Definition 2.2) are denoted using capital letters such as A, R, P, T, Q and for iteration space, IS. Presburger set $R_{x_1,x_2,\ldots,x_d}$, with set variables $(x_1, x_2, \ldots, x_d)$ and Presburger formula, P, is written as follows:

$$R_{x_1,x_2,\ldots,x_d} = \{[x_1, \ldots, x_d] | P\}.$$

Consider the dot product over two dense tensors in Figure 1, expressed in tensor notation in Figure 1(a) with corresponding C code in Figure 1(b). We describe the *iteration space* for the

$$v() = A(i) * B(i)$$

(a) Dot product

```
1 for (int i = 0; i < N; ++i)
2     v+=A[i]*B[i];
```

(b) Dot product between two dense vectors

$$IS = \{[i] | 0 \leq i < N\}$$

(c) Iteration space

$$T = \{[i] \rightarrow [Ti, ti] \mid ti = i \wedge 0 <= 8Ti < N \wedge 8Ti <= ti < 8Ti + 8\}$$

(d) Tiling transformation

$$IS = \{[Ti, ti] \mid 0 <= 8Ti < N \wedge 0 <= ti < N \wedge 8Ti <= ti < 8Ti + 8\}$$

(e) Tiled iteration space

```
1 for (int Ti=0; 8*Ti<N; ++Ti)
2   for (int ti=8*Ti; ti<min(8*Ti+8,N); ++ti)
3     v+=A[ti]*B[ti];
```

(f) Tiled code

Fig. 1. Dot product between dense vectors.

statement on line 2 by the polyhedron of Figure 1(c).[1] A statement macro is used to represent the statement at line 2 as a function of loop index i.

An important capability of polyhedral frameworks is the ability to represent transformations on loop nests as affine mappings on the iteration spaces. For example, tiling the i loop iteration by 8 can be represented by Figure 1(d). When T is applied to the iteration space of Figure 1(c), the resulting iteration space is Figure 1(e). A sequence of transformations are applied by composing the mappings. Polyhedral compilers generate code by performing polyhedral scanning [5, 17, 56]. Scanning produces constraints on each loop index from the iteration space description. These constraints are directly translated to for loops and if conditions in the generated code. Loop indices in the transformed statement of Line 2 are substituted using the inverse mapping resulting in code shown in Figure 1(f).

Some of the common operations on Presburger sets and relations are used in this article.

Definition 2.1. Intersection between Presburger sets, $R = R_1 \cap R_2$:

$$\mathbf{s} \in R \iff \mathbf{s} \in R_1 \wedge \mathbf{s} \in R_2.$$

Definition 2.2. A Presburger relation denotes a binary relation between the input set of indices, $\mathbf{i}$, and output set of indices, $\mathbf{o}$, described as $R_{\mathbf{i} \rightarrow \mathbf{o}} = \{\mathbf{i} \rightarrow \mathbf{o} | P_{\mathbf{i} \rightarrow \mathbf{o}}\}$.

Definition 2.3. Compositions are between two Presburger relations, $R_{\mathbf{x} \rightarrow \mathbf{o}} = R_{\mathbf{i} \rightarrow \mathbf{o}} \circ R_{\mathbf{x} \rightarrow \mathbf{i}}$:

$$\mathbf{x} \rightarrow \mathbf{o} \in R_{\mathbf{x} \rightarrow \mathbf{o}} \iff \exists \mathbf{i}\ s.t.\ \mathbf{i} \rightarrow \mathbf{o} \in R_{\mathbf{i} \rightarrow \mathbf{o}} \wedge \mathbf{x} \rightarrow \mathbf{i} \in R_{\mathbf{x} \rightarrow \mathbf{i}}.$$

2.3 Sparse Polyhedral Framework

Polyhedral frameworks cannot directly represent sparse tensor computations, which exhibit non-affine subscript expressions and loop bounds. Figure 2 illustrates the dot product using two sparse vectors. Figure 3 shows the difference between a dense version, which computes the sum of pairwise products of all elements of two vectors, and the version that uses sparse vectors, where products are only computed when the corresponding element of both vectors is nonzero. Figure 2(a)

[1] Auxiliary indices may be introduced to differentiate different statements in the same loop level for imperfectly nested loop nests.

```
// Data structure definition
struct SpVec { int len; int *idx; double *val; };
// Kernel signature
void SparseDotProduct(double &v, SpVec &A, SpVec &B);
```

(a) Data input of sparse dot product as arguments to kernel function.

$$IS = \{[pA, pB, i] | A.idx(pA) = i = B.idx(pB) \wedge 0 \leq pA < A.len \wedge 0 \leq pB < B.len\}$$

(b) Iteration space.

```
1 for (int pA = 0; pA < A.len; ++pA)
2  i = A.idx[pA]; // i-loop degenerates into assignment
3  for (int pB = 0; pB < B.len; ++pB)
4   if (i == B.idx[pB])
5    v += A.val[pA] * B.val[pB];
```

(c) Dot product between two sparse vectors resulting from polyhedral scanning in SPF.

```
pB = 0;
for (int pA = 0; pA < A.len; ++pA) {
 i = A.idx[pA];
 while (pB < B.len && i > B.idx[pB]) ++pB;
 if (pB < B.len && i == B.idx[pB])
   { v += A.val[pA] * B.val[pB]; ++pB; }
}
```

(d) Example optimized code generated by our framework.

Fig. 2. Dot product between sparse vectors.

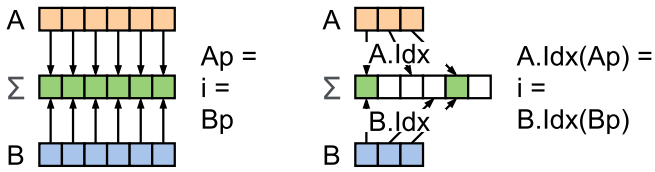


Fig. 3. Iteration space comparison between dense and sparse dot product. Coordinate index i is introduced to illustrate how nonzeros of the same coordinates are matched to produce result.

represents the *layout* of the input sparse vectors using the `struct SpVec`. The nonzero values are stored in `A.Val` and their coordinates are in `A.idx`. Because `A.idx` is used to encode coordinates of the nonzero values, it is commonly referred to as an *index array*. Accesses through `A.idx` introduce indirection and unknown bounds and conditions and are not in the affine domain.

To represent this computation, the **Sparse Polyhedral Framework (SPF)** introduces ***uninterpreted functions*** **(UFs)** in Presburger formulae to represent runtime values of index array references and other non-affine indices and loop bounds [70]. In SPF, uninterpreted functions $A.idx$ and $B.idx$ are used to describe the combined iteration space in Figure 2(b). In Figure 2(c), we can scan the points in this iteration space and use the condition at line 3 to ensure that both vector elements are nonzero before adding their product to the sum. The resulting code *co-iterates* over the common nonzero elements in the vectors. It requires a full sweep over B for every element of A so the time complexity of this code is $O(A.len * B.len)$.

Definition 2.4. (Uninterpreted function (UF)) An uninterpreted function f with arity of m, represents a mapping of $\mathbb{Z}^m \rightarrow \mathbb{Z}$.

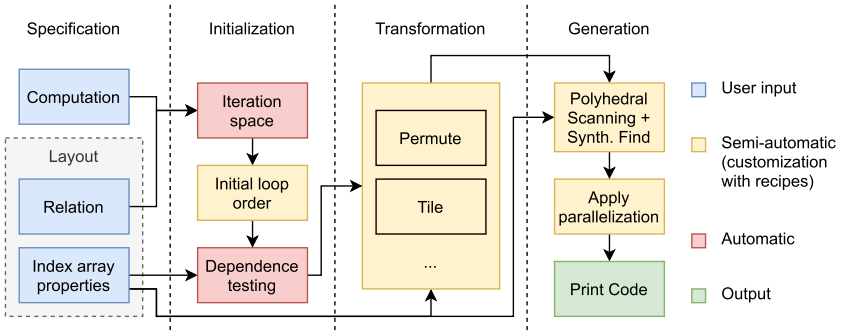


Fig. 4. Overview of the code generation process for sparse tensor contraction in our polyhedral framework.

In this work, UFs are used in Presburger formulae as another term type in the affine constraints, along with index variables. We allow arguments to the UFs to be an affine combination of integer set variables, integer constants, and other UFs. This ability enables polyhedral analysis to be performed on the UFs' arguments that can deduce (in-)equality relations on the arguments.

2.4 Overview of Approach

In this article, we optimize co-iteration using extensions to the Sparse Polyhedral Framework, thus producing the optimized code in Figure 2(d) with time complexity $O(A.len + B.len)$. There are two central ideas in this approach: (1) we compose the logical iteration space of the computation with the iteration space over the layout of the sparse tensors; and (2) the conditions of the co-iteration are derived using polyhedral scanning to iterate over the elements of one sparse tensor and look up corresponding elements in the other sparse tensors using a `find` algorithm. In Figure 2(d), `find` is implemented using the `while` loop and conditional, which represents a sequential iteration over ordered tensors that store their nonzeros in increasing coordinate order. Whether a find algorithm can be used is determined during code generation using an SMT solver, with the set of constraints arising from the layout specification.

Figure 4 illustrates the four stages of this approach. We begin with a specification of both the computation and layout, as described in Section 3. The second stage derives the iteration space of the computation, described in Section 4. In the third stage, we apply polyhedral transformations, as described in Section 5. The last stage generates the code using the polyhedral scanning of transformed iteration space combined with code synthesis of `find` using an SMT solver, as described in Section 6. The resulting code is then parallelized using OpenMP pragmas.

3 LAYOUT AS PHYSICAL-TO-LOGICAL RELATION

In this section, we describe how sparse layouts can be represented in a sparse polyhedral framework. The key focus of this article is to show that such a description can be incorporated into automated code generation of sparse tensor contraction. In practice, layouts can be described by compiler developers, library writers, or expert programmers, where end-users need not be directly exposed to these descriptions.

A *layout* is a *physical* ordering of the data in memory. Typically, a layout represents the nonzero tensor values and a collection of auxiliary *index arrays* that record coordinate information for the nonzeros to preserve the underlying *logical* view of the data. We define a relation $R_{\mathbf{p}\to\mathbf{g}}$ that maps nonzero elements in the sparse tensor representing the physical space $\mathbf{p}$ to their corresponding logical coordinates $\mathbf{g}$. In $R_{\mathbf{p}\to\mathbf{g}}$, index arrays are represented by uninterpreted functions by definition, because they contain read-only runtime values accessed through integer indices (arguments).

Table 1. Properties of Uninterpreted Function f Representing an Index Array that Has One Dimension a

Array property	Table 2 Layout	Definition
Range	BCSR	$MIN <= f \wedge f <= MAX$
Injectivity	Unsorted-COO	$a \neq a' \rightarrow f \neq f'$
(Strict) Monotonicity	SV/DCSR/Sorted-COO	$a \bowtie a' \rightarrow f \bowtie f'$
Periodic Monotonicity	CSR/DCSR/BCSR	$period(\mathbf{i}) \wedge a \bowtie a' \rightarrow f \bowtie f'$
Co-vary (w.g) Monotonicity	Sorted-COO	$g(\mathbf{i}) = g(\mathbf{i}') \wedge a \bowtie a' \rightarrow f \bowtie f'$

$\bowtie$ represents any order comparison conditional. Examples of these properties are present in the layouts described by Table 2, used in the experiments. Note that universal quantification, $\forall$, is assumed for $\mathbf{i}$, $\mathbf{i}'$, a, a'.

```
1 struct CSR {
2   int numRows;
3   int *rowPtr, *colIdx;
4   double *data;
5 };
```

(a) CSR data structure definition

$$R_{\mathbf{p}\rightarrow\mathbf{g}} = \{ [p_i, p_j] \rightarrow [g_i, g_j] | 0 \leq p_i < numRows \wedge rowPtr(p_i) \leq p_j < rowPtr(p_i + 1) \wedge g_i = p_i \wedge g_j = colIdx(p_j) \} : value = data[p_j]$$

(b) Physical-to-logical relation

$$p_i = p_i' \wedge a < a' \rightarrow colIdx < colIdx'$$

(c) Index array property: periodic monotonicity.

Fig. 5. Compressed Sparse Row (CSR) layout specification.

While values of index arrays cannot be determined until runtime, properties associated with their values are sometimes statically known and are useful to static optimization. Table 1 lists simplified versions of the index array properties we use, demonstrated with a single argument a. These properties are expressed using logical formulae with guard conditions on array indices and constraints on array values as in Bradley et al. [14], Mohammadi et al. [52]. The arguments to the uninterpreted functions can be affine combinations of constants, indices, and uninterpreted functions. Additionally, if there are non-affine expressions in the relation, then they too can be modeled as uninterpreted functions. Compared to the goal-oriented uses of index array properties in prior works, such as for disproving dependences [52], index array properties in this work are a component of the layout's description that targets the general question of code generation for sparse computation.

Figure 5(b) presents $R_{\mathbf{p}\rightarrow\mathbf{g}}$ for the common ***Compressed Sparse Row* (CSR)** layout declared as in Figure 5(a). Index array `rowPtr` refers to the first nonzero element of each row in the `val` vector of nonzeros. Index array `colIdx` refers to the column associated with each nonzero element. Note that $R_{\mathbf{p}\rightarrow\mathbf{g}}$ describes both the layout indices p_i, p_j for iterating over the sparse layout, and the logical indices g_i, g_j for iterating over a 2-D tensor A. As nonzero elements in CSR layout are sorted by row, a periodic monotonicity property exists for `colIdx` array, which is expressed by the logical formula in Figure 5(c). This logical formula denotes when two iteration instances containing $colIdx$ are induced from the same physical index $p_i = p_i'$. If the first instance's argument, a, is smaller than that of the second, a', then the first value, $colIdx$, will also be smaller than the second, $colIdx'$. Applying polyhedra scanning to this description, our compiler can generate the code in Figure 6, which iterates over all nonzeros in the layout.

Index arrays aid in providing coordinates corresponding to nonzeros in a sparse tensor, so the layout description can be specified as a mathematical relation from the data structure scalar and

```
1 for (int p_i = 0; p_i < csr.numRows; ++p_i)
2  for (int p_j = csr.rowPtr[p_i]; p_j < csr.rowPtr[p_i+1]; ++p_j) {
3   g_i = p_i;  g_j = csr.colIdx[p_j];  value = csr.data[p_j];
4   y[g_i] = value * x[g_j]; }
```

Fig. 6. Generated code that iterates over nonzeros in CSR and uses them to compute sparse matrix vector multiplication, $y(i) = A(i,j) * x(j)$. Indices g_i and g_j represent the logical indices associated with the *value*. The computation on Line 4 demonstrates how these logical indices can be used.

array fields to the tensor. By expressing layouts as polyhedral relations, tensor contraction operations with sparse tensors can be composed using polyhedral set and relation operations, as discussed in the following section.

4 DERIVING ITERATION SPACE FROM COMPUTATION AND LAYOUT

A sparse tensor contraction computation may involve two or more sparse tensors, potentially using different layouts. This section describes how such layouts, specified using sparse polyhedral relations, can be combined with the access pattern in the computation to derive an iteration space. This enables subsequent transformations to the iteration space.

As an illustrative example, consider the tensor contraction matrix multiplication. In tensor index notation, this contraction is written as $C(i,j) = A(i,k) * B(k,j)$. The following iteration space results for loop indices $\mathbf{I} = [i,k,j]$:

$$IS_{\mathbf{I}} = \{[i,k,j] : 0 \leq i < N \wedge 0 \leq j < N \wedge 0 \leq k < N\}.$$

We represent an access expression for tensors C, A, and B as a mapping from the computation's iteration space $\mathbf{i}$ to the tensor's data space:

$$A^{(C)}_{\mathbf{I}\to\mathbf{g}} = \{[i,k,j] \to [g_i,g_j] | g_i = i \wedge g_j = j\},$$
$$A^{(A)}_{\mathbf{I}\to\mathbf{g}} = \{[i,k,j] \to [g_i,g_j] | g_i = i \wedge g_j = k\},$$
$$A^{(B)}_{\mathbf{I}\to\mathbf{g}} = \{[i,k,j] \to [g_i,g_j] | g_i = k \wedge g_j = j\}.$$

To determine the part of logical iteration space $\mathbf{I}$ that accesses nonzeros in the sparse tensor representation, we derive $Q_{\mathbf{p}\to\mathbf{I}}$: the composition of the layout description $R_{\mathbf{p}\to\mathbf{g}}$ with the access mapping $A_{\mathbf{I}\to\mathbf{g}}$. Using CSR layout for A as described in Figure 5(c), we have the relation as follows:

$$\left(A^{(A)}_{\mathbf{I}\to\mathbf{g}}\right)^{-1} \circ R^{(A)}_{\mathbf{p}\to\mathbf{g}} =$$
$$Q^{(A)}_{\mathbf{p}\to\mathbf{I}} = \{\ [p_i,p_j] \to [i,k,j] | 0 \leq p_i < numRows \wedge$$
$$rowPtr(p_i) \leq p_j < rowPtr(p_i+1) \wedge$$
$$i = p_i \wedge k = colIdx(p_j)\}.$$

Definition 4.1. Range of a Presburger relation, $R_{\mathbf{o}} = Range(R_{\mathbf{i}\to\mathbf{o}})$:

$$\mathbf{o} \in R_{\mathbf{o}} \iff \exists i \ s.t. \ \mathbf{i} \to \mathbf{o} \in R_{\mathbf{i}\to\mathbf{o}}.$$

$Range(Q^{(A)}_{\mathbf{p}\to\mathbf{I}})$ corresponds to all positions in the iteration space of $\mathbf{i}$ that have a nonzero or explicit zero stored in the sparse format of A.

The new iteration space accessing multiple sparse tensor layouts will be the intersection or union of the parts of the original iteration space that accesses sparse tensors based on whether the computation is a multiplication (intersection) or an addition (union). This is as a sparse polyhedral definition of merge lattices proposed by TACO [34, 41]. When multiple tensors are multiplied, such

as $A(i,k) * B(k,j)$ in the example, the value may be nonzero if and only if both sparse tensors store the value for the iteration (i,k,j). Thus, intersection is used to combine layouts relations under the logical access of the computation in multiplication, P for product:

$$P = Range\left(Q^{(A)}_{\mathbf{p}\to\mathbf{I}}\right) \cap Range\left(Q^{(B)}_{\mathbf{p}\to\mathbf{I}}\right).$$

The iteration space for transformation and code generation also includes the layout for the output tensor and the dense iteration space. In our implementation, we support dense output tensors as well as sparse tensors with known sparsity. Under such cases, the output of the tensor contraction can be treated as another product term and intersected with P. Known output sparsity is common in core computations of data analytics [16] and **graph neural network (GNN)** [80] such as **Sampled Dense-Dense Matrix Multiplication (SDDMM)** and **Sparse Matrix Times Dense Vector (SpMM)**, and can be generated for other sparse computations using inspectors. The dense iteration space represents optional bounds that can bound the computation to a sub-region of the valid iterations specified by the layouts, such as when a computation only operates on the lower triangular region of a layout. Further bound by the dense iteration space, we have the iteration space for polyhedral transformation and code generation:

$$IS = P \cap Range\left(Q^{(C)}_{\mathbf{p}\to\mathbf{I}}\right) \cap IS_{\mathbf{I}}.$$

Note that the existential operation on the input indices in the definition for the range operations (Definition 4.1) does not guarantee the input indices can be eliminated through simplification. In fact, with $Q^{(A)}_{\mathbf{p}\to\mathbf{I}}$, the input indices hold special meaning in the iteration space and may reference index arrays. Some of these indices not only have to be "rematerialized" during the code generation but can also be involved in transformations like positional tiling [61].

Instead of relying on the code generator to make decisions when to rematerialize existential variables, we pull these indices out of the existential operations and make them part of the set variables of the iteration space. Thus, the iteration space will consist of all layout indices and the indices of the computation:

$$\begin{aligned}
IS &= P \cap Range\left(Q^{(C)}_{\mathbf{p}\to\mathbf{I}}\right) \cap IS_{\mathbf{I}} \\
&= Range\left(Q^{(A)}_{\mathbf{p}\to\mathbf{I}}\right) \cap Range\left(Q^{(B)}_{\mathbf{p}\to\mathbf{I}}\right) \cap Range\left(Q^{(C)}_{\mathbf{p}\to\mathbf{I}}\right) \cap IS_{\mathbf{I}} \\
&= \{[\mathbf{I}] | (\exists \mathbf{p}^{(A)} \ldots) \wedge (\exists \mathbf{p}^{(B)} \ldots) \wedge (\exists \mathbf{p}^{(C)} \ldots) \wedge \ldots\} \\
&= \{[\mathbf{I}] | (\exists \mathbf{p}^{(A)} (\exists \mathbf{p}^{(B)} (\exists \mathbf{p}^{(C)} \ldots \wedge \ldots \wedge \ldots \wedge \ldots)))\} \\
&= \{[\mathbf{p}^{(A)}, \mathbf{p}^{(B)}, \mathbf{p}^{(C)}, \mathbf{I}] | \ldots \wedge \ldots \wedge \ldots \wedge \ldots\}.
\end{aligned}$$

For additions, union can be used, such as, for $v() = A(i) + B(i)$, S for sum, $S = Range(Q^{(A)}_{\mathbf{p}\to\mathbf{I}}) \cup Range(Q^{(B)}_{\mathbf{p}\to\mathbf{I}})$. However, union will cause implicit zeros of one of the tensors being accessed when some other tensors are not zero. Alternatively, the polyhedral framework can use statement splitting to give each addition term its own iteration space to guard their execution. For example, $v() = A(i) + B(i)$ can be split into $v() = A(i)$ and $v() = B(i)$. Either approach benefits from additional optimizations to save space or fuse separate loops, outside the scope of this article.

5 POLYHEDRAL ANALYSIS & TRANSFORMATIONS

In the previous section, we demonstrated how to combine relations defined by the layouts and the iteration space of the computation to form the iteration space of the generated code. In this section, we show how using a sparse polyhedral representation permits reasoning about parallelism and

```
struct DCSR {
    int numRows;
    int *rowIdx, *rowPtr;
    int *colIdx;
    double *data;
};
```

(a) Data structure definition.

$$R_{\mathrm{p}\to\mathrm{g}} = \{\ [p_i, p_j] \to [g_i, g_j] | 0 \le p_i < numRows \wedge$$
$$rowPtr(p_i) \le p_j < rowPtr(p_i + 1) \wedge$$
$$g_i = rowIdx(p_i) \wedge g_j = colIdx(p_j)\}$$
$$: value = data[p_j]$$

$$a < a' \to rowIdx < rowIdx'$$
$$p_i = p_i' \wedge a < a' \to colIdx < colIdx'$$

(b) Layout definition.

```
void spmv(double *y, DCSC A, double *x) {
 for (int p_i = 0;p_i < A.numRows; ++p_i)
  for (int p_j = A.rowPtr[p_i]; j < A.rowPtr[p_i+1]; ++p_j)
   y[A.rowIdx[p_i]] += A.data[p_j] * x[A.colIdx[p_j]];}
```

(c) Sparse matrix vector multiplication, $y(i) = A(i, j) * x(j)$.

Fig. 7. Doubly compressed sparse row (DCSR).

composing code transformations. These concepts are exemplified with discussions on dependence testing and tiling.

5.1 Dependence Testing

In general, dependence testing determines if it is safe to parallelize a loop or apply a transformation, realized with a *dependence polyhedron* [27, 76] in polyhedral frameworks. Tensor contraction expressions exhibit specific data dependence patterns. In matrix vector multiplication, $y(i) = A(i, j) * x(j)$, j is a contraction index that carries reduction dependences, since $y(i)$ is the sum over $A(i, j) * x(j)$ products; i is a free index without loop-carried dependences. Since the generated code also iterates over the layout indices, the compiler must translate dependence relations to refer to layout indices, which can be described in the sparse polyhedral framework.

We derive the dependence polyhedron for two accesses to the same tensor, where one of them is a write, using the combined iteration space of Section 4, and the lexicographical loop order. Because the sparse layout will not change the inherent dependences of the computation, we observe dependences from the logical access, allowing us to circumvent complexities arises from performing dependence analysis on the value arrays through indirect accesses with index arrays in the sparse layout. This dependence polyhedron can be also combined with the dense dependences to determine whether the original order of the computation is preserved.

Index array properties added to the dependence polyhedron allow dependence testing to be more precise, as shown with the ***Doubly Compressed Sparse Row* (DCSR)** layout in Figure 7. Consider the `p_i` loop in Figure 7(c); the information that $A.rowIdx$ is monotonically increasing proves that loop `p_i` does not carry a dependence.

5.2 Tiling

We can express transformations such as tiling as relations on the iteration space, such as in the example of Figure 8. The combined iteration space of Section 4 guards execution in Figure 8(a) based on the value of a UF f representing an index array. Tiling transforms the loop into loops on Line 1 and 3 of Figure 8(c). However, the compiler introduces the condition on Line 2 due to the monotonicity of f as an optimization after tiling is applied. This introduced condition will significantly reduce the number of tiles executed and improves the performance.

```
1 for (i = 0; i < n; ++i)
2  if (f[i] == k) S0();
```

(a) Original program where k is a loop invariant value in i-loop.

$$a < a' \rightarrow f < f'$$

(b) Monotonicity of f.

```
1 for (ti = 0; 8*ti < n; ++ti)
2  if (f[8*ti]<=k && f[min(n, 8*ti + 8) - 1]>=k)
3   for (i = 8*ti; i < min(n, 8*ti + 8); ++i)
4    if (f[i] == k) S0();
```

(c) Tiled computation with induction loop ti. Line 2 demonstrates the constraints introduced by monotonicity of f.

Fig. 8. Tiling a sparse computation with UF properties.

To achieve sparse tiling, these constraints are generated from matching the UF property expression with bounds produced from the polyhedral scanning.

6 CODE GENERATION

With the combined iteration space in Section 4, we can generate code that iterates over the parts described by the layout indices—subsections, rows, columns, or nonzeros—of one or more sparse tensors and look up corresponding parts in the next sparse tensor in its layout indices using *find* algorithms, as described in this section.

6.1 Co-iteration Using Polyhedral Scanning

Polyhedral scanning can generate loops that handle co-iteration with if conditions from the iteration space constraints, such as IS in Figure 2(b) for the sparse-sparse vector dot product. Such code is generated through classic polyhedral scanning algorithms [18], where conditions involving each loop index are produced through set operations. Loops are generated from these conditions by extracting the lower bounds and upper bounds from the index conditions. Conditions other than bounds are turned into stride when they specify modulo equality and if conditions if otherwise. For example, the polyhedral scanning produces the following constraint for the pB loop:

$$0 \leq pB \wedge pB < B.len \wedge i = B.idx(pB). \tag{1}$$

Note that in this relation, the first two terms specify the loop bound, and the third term specifies the if condition.

The resulting code is shown in Figure 2(c) with two loops, one for each layout—pA and pB—and a condition in the pB loop that relates locations in these two sparse layouts. This code will work regardless of whether the elements of either vector are sorted. However, more efficient code can be generated when it is known that the vectors are sorted—discussed next.

6.2 Optimized Co-iteration Using Synthesis of Find

The conditions specified in 1 that produce the pB loop at line 3 and if condition at line 4 in Figure 2(c) can be alternatively described as a find: looking for index pB within the loop bound such that $B.idx(pB) = i$. When it is treated as a `find(B,pB)`, different find algorithms can be used to replace this loop. We illustrate two examples of find algorithms that replace the pB loop in Figure 9, `SeqIter`, which refers to sequential iteration, and `HashMap`. `SeqIter` find matches by scanning through the loop range of pB using an inequality version of the find condition until a match is found or no matches are possible. While the scanning is linear, the initialization of pB affects if scanning resumes from the last saved position. Through amortization, `SeqIter` is of complexity

```
pB=0;

for (int pA=0;pA<A.len;++pA) {
 i = A.idx[pA];
 while(pB<B.len&&i>B.idx[pB]) ++pB;
 if(pB<B.len&&i==B.idx[pB])
  { v+=A.val[pA]*B.val[pB]; ++pB; }}
```

(a) Sequential iteration, `SeqIter`.

```
for (int pB=0;pB<pB.len;++pB)
  hashB[B.idx[pB]] = pB;
for (int pA=0;pA<A.len;++pA) {
  i = A.idx[pA];
  pB = hashB.find(i)
  if (pB!=hashB.notfound)
    v+=A.val[pA]*B.val[pB];}
```

(b) `HashMap`.

Fig. 9. Find algorithms and sparse vector dot product. Code belonging to the templates is highlighted.

ALGORITHM 1: Augmented polyhedral scanning.

Input: *IS*: Extended Iteration Space. *Idx*: Loop indices in combined iteration space *IS* from outermost to innermost.

for *Index* $i \in Idx$ **do**
  L_i = scan iteration ranges of i in IS ; `/* Polyhedral */`
  UF = uninterpreted functions applied with i as the last unbound index;
  $C_i = map\langle UF, bounds\rangle$;
  for $uf \in UF$ **do**
    if *Scan equality/range R of uf in IS* **then** `/* Polyhedral */`
      Insert (uf, R) in C_i;
  if *Find algorithm A can be applied on* L_i, C_i **then** `/* SMT */`
    Generate A;
  else
    Generate `for` loop with bound L_i and `if` condition C_i;

$O(A.len+B.len)$. `HashMap` uses a hashmap to perform the find. It can only handle equality find conditions and has a complexity of $O(A.len + B.len)$, including the initialization cost of the hashmap.

Each find algorithm has a basic skeleton of code associated with it. This skeleton is captured in the code generator with a template for each algorithm. During code generation, the find algorithm's template will be filled in with constraints arising from the computation, index array properties, and loop permutation order: They can be integer values or algebraic expressions. Assumptions of each algorithm determine which templates are valid and how to generate the template arguments. For example, the `SeqIter` code in Figure 9 is only valid when the elements of each vector are sorted. For a given find algorithm, its assumptions are encoded using logical formulae. The generation of the find algorithm will try to match the assumptions with conditions from iteration spaces, index array properties, and generated template arguments.

We use a ***satisfiability modulo theory* (*SMT*)** solver to prove if these assumptions are met. Template arguments are first generated through enumeration or construction, and then the assumptions are checked with the generated arguments. This method of generating code segments by proving generated code with an MT solver or **high-order logic (HOL)** provers is commonly referred to as *code synthesis* [3, 66]. The details of this synthesis process are presented as supplemental material.

6.3 Code Generation Algorithm

We present the tensor contraction code generation algorithm in Algorithm 1, which augments polyhedral scanning to leverage an SMT solver to synthesize find algorithms. Each loop index in

the combined iteration space *IS* from Section 4 is processed from outermost to innermost. Line 3 identifies all uninterpreted functions that are fully bound at this loop level. Lines 5–7 extract find conditions produced by the combined iteration space. Lines 8–9 use the SMT solver to detect and generate find algorithm *A* as an alternative to the loop and if conditions.

Applying Algorithm 1 on the iteration space of sparse dot product in Figure 2(b), polyhedral scanning is used to identify the range for loop *pA*. There are no uninterpreted functions at this loop level, and a *for*-loop is generated. For the next loop *pB*, $A.idx(pA)$ is loop-invariant. When $A.idx$ and $B.idx$ are monotonically increasing, SMT solver can prove that find algorithms such as *sequential iteration* and *hashmap* can be applied to implement the find in *pB*. When *sequential iteration* is applied, the code in Figure 2(d) is generated. Loop *i* will be generated as an assignment from scanning, $i = A.idx(pA)$ and subsequently removed by dead code elimination due to no usage related to *i*.

7 DEMONSTRATIONS AND COMPARISONS

In this section, we demonstrate our proposed framework in two aspects: the versatility of our sparse layout specification and the adaptability of our code generation strategy.

7.1 Sparse Tensor Layouts

Table 2 presents sparse tensor layouts as described in our framework using the approach in Section 3. All but the last layout are used in the experimental evaluation. The first column describes the layout using common terms or citations. Without loss of generality, higher-order sparse tensor layouts such as **Compressed Sparse Fiber (CSF)** can be similarly specified with relations including uninterpreted functions representing index arrays and logical formulae describing index array properties.

In the table, the last column compares how these layouts are supported by the TACO compiler [41]. TACO uses level format and mode ordering to specify how and in what order dimensions are stored. Four level formats are defined: dense, compressed, sparse, and singleton. Mode ordering specifies the order in which levels are organized. Each level format can be further customized with properties such as uniqueness and sortedness. Two points of difference with TACO are (1) its coupling of logical dimensions to the physical dimensions, thus disallowing a logical dimension to be derived from multiple physical dimensions[2]; and (2) TACO views each dimension separately, thus disallowing relations such as $col \leq row$ in the lower-triangular matrix.

Looking to the future of coarse-grained functional units such as the NVIDIA A100 sparse tensor core, we show how our approach describes the Warp Sparse Matrix Storage [2, 51]. Our code generation can produce computation kernels on the host CPU for computations not natively supported by the tensor core without requiring layout changes or writing complex architecture-specific code.

7.2 Comparison with Conjunctive Merge

This subsection compares the code generated by our approach as compared with that of the TACO compiler [41], using sparse dot product as an example. Specifically, Figure 10(a) revisits the code generated by our sequential iteration algorithm template, where the `i` loop at line 2 iterates over the nonzero elements in the layout of *A*, and the loop at line 4 along with the condition at line 6 looks for that element in *B*. It only examines each element of *A* once and then searches adjacent elements in *B* for the index in *A*, only visiting an element in *B* twice if it matches an element in *A*. Our approach can alternatively generate code with loop permutation, which iterates over

[2]BCSR, defined in Table 2, is not possible with TACO due to the relation on g_j. TACO can describe a more restricted version of BCSR with aligned $g_j = 8 * colIdx(p_j) + p_j$: D,C(unique),D,D.

Table 2. Different Layout Definitions

Data structure definition	Layout definition
Sparse Vector (SV) [C(unique)]	
`struct SpVec { int len; int *idx; double *val; };`	$R_{\mathrm{p;g}} = \{ [p_i] \rightarrow [g_i] \mid 0 \le p_i < len \wedge g_i = idx(p_i) \}$: $value = val[p_i]$ $a < a' \rightarrow idx \le idx'$
Compressed Sparse Row (CSR) [60] [D,C(unique)]: Figure 5	
Doubly Compressed Sparse Row (DCSR) [15] [C(unique),C(unique)]: Figure 7	
Coordinate (COO) [60] [C(not-unique),Q]	
`struct COO { int numNNZ; int *rowIdx; int *colIdx; double *data; };`	$R_{\mathrm{p;g}} = \{ [p_i] \rightarrow [g_i, g_j] \mid 0 \le p_i < numNNZ \wedge g_i = rowIdx(p_i) \wedge g_j = colIdx(p_i) \}$: $value = data[p_i]$ $a < a' \rightarrow rowIdx \le rowIdx'$ $a < a' \wedge rowIdx(a) = rowIdx(a') \rightarrow colIdx < colIdx'$
Block Compressed Sparse Row (BCSR) [35]	
`struct BCSR { int numRows; int *rowPtr; int *colIdx; double *data[8][8]; };`	$R_{\mathrm{p}\rightarrow\mathrm{g}} = \{ [p_i, p_j, p_k, p_l] \rightarrow [g_i, g_j] \mid 0 \le p_i < numRows \wedge rowPtr(p_i) \le p_j < rowPtr(p_i + 1) \wedge 0 \le p_k < 8 \wedge 0 \le p_l < 8 \wedge g_i = p_i * 8 + p_k \wedge g_j = colIdx(p_j) + p_l \}$: $value = data[p_j][p_k][p_l]$ $p_i = p_i' \wedge a < a' \rightarrow colIdx + 8 \le colIdx'$
Lower triangular	
`struct LowerTri { int numRows; double *data; };`	$R_{\mathrm{p}\rightarrow\mathrm{g}} = \{ [p_i, p_j] \rightarrow [g_i, g_j] \mid 0 \le p_i < numRows \wedge 0 \le p_j \le p_i \wedge g_i = p_i \wedge g_j = p_j \}$: $value = data[p_i * (p_i + 1)/2 + p_j]$
Warp Sparse Matrix Storage [2, 51]	
`struct CUDAmma16x16 { float data[16][8]; Bits<512> offset; };`	$R_{\mathrm{p}\rightarrow\mathrm{g}} = \{ [p_i, p_j] \rightarrow [g_i, g_j] \mid 0 \le i < 16 \wedge 0 \le p_j < 8 \wedge g_i = p_i \wedge g_j = p_j * 2 + offset(p_i * 32 + p_j * 4 + 1) \}$: $value = data[p_i][p_j]$ $0 \le offset < 2$

TACO's layout description is shown in square bracket when possible using the level formats, dense (D), compressed (C), and singleton (Q), with properties in the parentheses per level.

B and performs a lookup of A. The code in Figure 10(a) will perform better when A contains fewer nonzeros, since each element of A is only examined once, and vice versa for the permuted code. By comparison, TACO's *conjunctive merge algorithm* (Figure 10(b)) iterates over both sparse tensors in the while loop at line 2 and must examine an element of A and an element of B in each iteration, even if it was examined in the previous iteration. Note that all implementations have linear complexity $O(A.len + B.len)$, but the code generated by TACO exhibits more data movement.

The second difference relates to how we can handle a much greater set of index array properties than TACO. This is related to both the expressiveness of the layout description and the adaptivity

```
pB = 0;
for (int pA=0;pA<A.len;++pA) {
 i=A.idx[pA];
 while (pB<B.len&&i>B.idx[pB])
  ++pB;
 if (pB<B.len&&i==B.idx[pB])
 { v+=A.val[pA]*B.val[pB]; ++pB; }}
```

(a) Sequential iteration.

```
pA = 0; pB = 0;
while (pA < A.len && pB < B.len) {
 A0 = A.idx[pA]; B0 = B.idx[pB];
 i = min(A0,B0);
 if (A0 == i && B0 == i)
  v+=A.val[pA]*B.val[pB];
 pA += (int)(A0 == i);
 pB += (int)(B0 == i);}
```

(b) Conjunctive merge from TACO.

Fig. 10. Sparse vector dot products.

```
hashmap hashC;
for (pC = 0; pC < C.len; ++pC) hashC[C.idx[pC]] = pC;
pB = B.len - 1;
for (int pA = 0; pA < A.len; ++pA) {        // pA
 i = A.idx[pA];
 while (pB < B.len && i > B.idx[pB]) --pB; // pB sequential iteration
 if (pB < B.len && i == B.idx[pB]) {
  pC = hashC.find(i);                       // pC hashmap
  if (pC != hashC.notfound) v += A.val[pA] * B.val[pB] * C.val[pC];
  --pB;
 }}
```

Fig. 11. Three-way co-iteration, computing $v = A(i) * B(i) * C(i)$, where A has uniqueness and increasing monotonicity, B has uniqueness and decreasing monotonicity, and C has uniqueness but no monotonicity.

of the code generation. Figure 11 demonstrates a three-way co-iteration where each sparse vector involved has a different ordering on the nonzeros. TACO uses flags to specify index array properties. TACO thus can not express slight variations of the properties, such as decreasingly sorted used by B. When specific properties such as sortedness are not provided and the *locate*[3] level-function is not defined on a level format, TACO will also fail to generate code as in the case of C, which can be described as a compressed level format without sortedness. In our framework, synthesis allows more adaptability regarding variations of index array properties. Different find algorithms, including the fallback loop implementation, allows us always to generate valid and efficient code under the constraints provided.

8 EXPERIMENTS

We have implemented a polyhedral compiler with the layout specification, dependence testing, and sparse polyhedral code generation extensions presented in this article. In our implementation, we used functionalities provided by the CHiLL compiler [18], the Omega+ Library [17] for integer set manipulation and scanning, and Z3 [25] for theory proving.

8.1 Experiment Setup

The layouts in Table 2 can be used in any contraction computation expressed in tensor index notation. Because this work focuses on combining layouts and computation, we demonstrate these layouts in the context of the computations in Table 3. **Sparse matrix vector multiply (SpMV)** demonstrates sparse-dense co-iteration and provides the baseline performance of the generated

[3]Finding position from coordinate.

Table 3. Computations Used in Comparison

Name	Notation	Matrix size
SpMV	$y(i) = A(i,j) * x(j)$	SuiteSparse
SpMSpV	$y(i) = A(i,j) * x(j)$	SuiteSparse
SpMSpM	$C(i,j) = A(i,k) * B(k,j)$	Random 5k-5k
TTM (mode 1)	$C(i,j,l) = A(i,j,k) * B(k,r)$	Various Real-world Tensors

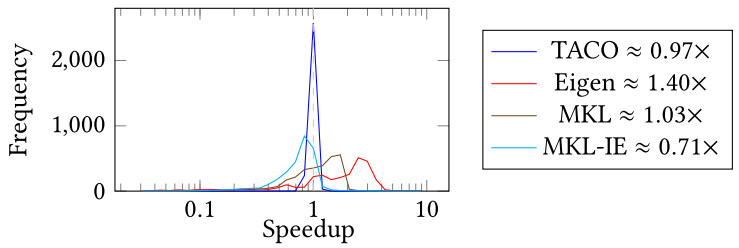


Fig. 12. Distribution of relative multi-threaded SpMV execution time on SuiteSparse matrices: We are able to generate code of similar quality when sparse-sparse co-iteration is not concerned. MKL-IE is the inspector executor version of MKL. Each bucket is of size 1.2: bucket x is $[1.2^{x-0.5}, 1.2^{x+0.5})$; the bucket around 1—no speedup—is $[0.913, 1.095)$.

code. SpMSpV deals with sparse-sparse co-iteration where only a single vector is co-iterated with each row. SpMSpM showcases the composability of our polyhedral-framework-based methods when multiple sparse layouts are involved, which results in complex loop conditions. We also added **tensor-times-matrix (TTM)** where the tensor is stored in **compressed sparse fiber layout (CSF)** [65].

These sets of experiments were run on a single-socket AMD EPYC 7702P CPUs at 2.0 GHz. This CPU exposes 4 NUMA domains corresponding to the 4 quadrants, each containing 16 cores, and has its own DRAM controller. Experiments measure multi-threaded code bound to one of the NUMA domains using the `numactl` utilities to prevent adverse NUMA effects from suboptimal thread placement.

The generated code is automatically parallelized by inserting the OpenMP pragma, `#pragma omp parallel for schedule (dynamic,32)`, at the outermost parallel loop based on static dependence testing. All code is compiled with GCC 10.2.0, with flags `-O3 -ffast-math -march=native`.

We compared the generated code with corresponding kernels from a state-of-the-art tensor algebra compiler—TACO [41], an optimized binary library—the Intel Math Kernel Libraries [36] 2021.1.1, a template library—Eigen [33] 3.3.7, and a state-of-the-art sparse linear algebra library—**SuiteSparse:GraphBLAS (SS:GB)** 5.10.1 [23]. Parallel implementations from the libraries are used when available. TACO is parallelized using the same OpenMP pragma to eliminate any difference arising from parallelization.

8.2 Performance without Co-iteration: SpMV

Figure 12 provides the performance comparison on 2,893 of the the real and pattern matrices in the SuiteSparse matrix collection [24] in CSR layout for all libraries. Each experiment is run at least two times or until 30 seconds have elapsed. The average speedup is reported by geometric mean over the speedup from all random sets.

Considering SpMV, we see comparable performance to TACO and the library implementations other than MKL-IE, demonstrating that code generation using polyhedral scanning and synthesis

Table 4. We Achieved Significant Speedup (Geometric Mean) on SpMSpV where Sparse-sparse Co-iteration Is Involved

	TACO	Eigen	SS:GB
`SeqIter`	1.63	2.43	1.50
`HashMap`	1.63	2.44	1.50
`Auto`	2.72	4.06	2.50

We present results for sequential iteration (`SeqIter`) or hashmap (`HashMap`) as find algorithms. `Auto` selects the best performance using these two algorithms.

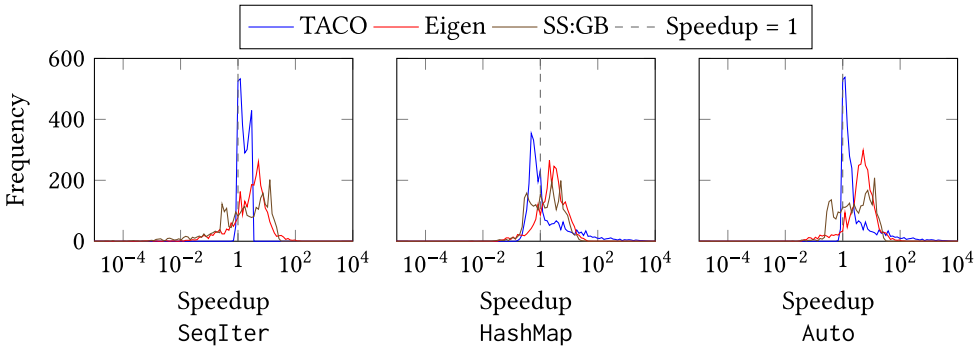


Fig. 13. Distribution of relative speedup of generated code to different libraries: `Auto` is consistently faster than other libraries. The right side of 1 indicates the implementation is able to obtain a speedup greater than 1×. Frequency denotes the number of matrices from SuiteSparse that have relative speedup in the bucket. Each bucket is of size 1.2: bucket x is $[1.2^{x-0.5}, 1.2^{x+0.5})$; the bucket around 1—no speedup—is $[0.913, 1.095)$.

achieves efficient code. MKL-IE achieves higher performance using a runtime inspector that can fine-tune the loop schedules and parallelization strategy of the executor; inspection time is not included in the execution time measurement.

8.3 Performance of Co-iteration: SpMSpV

`SpMSpV` demonstrates the performance of generated code on sparse-sparse co-iteration. We exclude 17 of the matrices due to either out-of-memory issues caused by SS:GB when performing inspections or time-out (longer than four hours) caused by TACO that also affects the sequential iteration to a lesser degree. The geometric mean speedup achieved is shown in Table 4 with the distribution of speedup shown in Figure 13.

For the individual find algorithms, we are able to achieve consistent performance improvement over the libraries. `SeqIter` is consistently faster than the comparable conjunctive merge algorithm from TACO, where both have an algorithm complexity of $O(rows{*}x.nnz{+}A.nnz)$. nnz stands for the number of nonzeros in the respective tensor. This improvement is from reduced data movement, as discussed in Section 7.2. Meanwhile, `HashMap` has an algorithm complexity of $O(A.nnz + x.nnz)$, which is much more efficient when the vector is denser than the matrix. The density is defined as the ratio of nonzeros over the size of the tensor in the logical space. Due to different complexity, `HashMap` achieves the largest speedup of 5,214× compared to TACO on the DIMACS10/europe_osm matrix, but its speedup is also less consistent. Eigen and **SuiteSparse:GraphBlas (SS:GB)** libraries

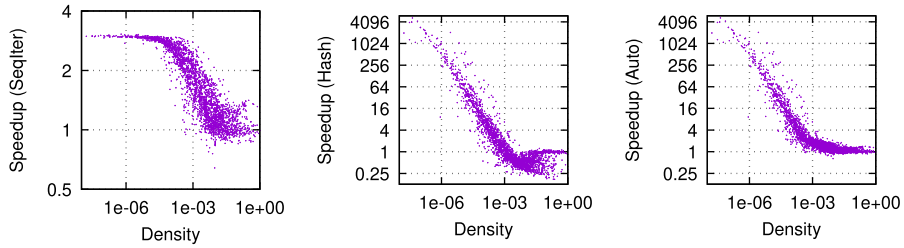


Fig. 14. Speedup against TACO under different matrix density. Due to less data movement, `SeqIter` is almost consistently more efficient than TACO's conjunctive merge. `HashMap` is of different algorithmic complexity compared to both TACO and `SeqIter`. By offering the flexibility of generating either algorithms, `Auto`, our code generation algorithm achieves non-constant speedup against TACO.

use computation methods that have a more stable algorithmic complexity related to the number of nonzeros in the sparse matrix, which is comparable to our generated code with `HashMap`. Furthermore, due to the flexibility of generating different find algorithms, we are able to achieve even higher speedup by selecting the best-performing algorithm at runtime.

As discussed, density is an important metric that affects the relative performance of different find algorithms. Figure 14 demonstrate its effects on different algorithms when compared to TACO. `SeqIter` is able to achieve a bounded speedup when the density of the matrix is low, up to 3.05×. Meanwhile, `HashMap` is less affected by the density of the sparse vector. It can achieve unbounded speedup proportional to the differences in density of the sparse matrix and the sparse vector's density of 0.1. By selecting the best-performing algorithm, we are able to achieve consistent speedup by avoiding the slowdowns from `HashMap` when the matrix density is high and improving the speedup potential when the matrix density is low.

8.4 Composing Programs with Multiple Layouts: SpMSpM

We next consider `SpMSpM` and analyze performance of co-iteration when the two sparse matrices have different layouts. As we want to look at layouts beyond CSR, and neither Eigen nor SS:GB support other layouts, this comparison is only with TACO. We used randomly generated matrices from TACO[3] with a fillrate 0.1 in each dimension. We report a geometric mean over 100 sets of random tensors in each experiment, where each set is run twice to avoid a cold start and run at least eight times and at least 5 seconds.

Table 5 demonstrates that our approach is able to compose complex layouts with computation while achieving comparable performance to TACO on computations that TACO supports. Our approach improves upon TACO's performance on COO by avoiding multiple sweeps over the sparse tensors. By decoupling the iteration over logical and sparse indices, our approach is able to support triangular layout and computations and blocked layouts such as BCSR. To support BCSR in TACO, due to the requirement of matching dimensions to iterators, the BCSR layout must be expressed as a fourth-order tensor that is incompatible with the other layouts besides BCSR [41].

8.5 Performance of Higher-order Tensors and Sharing of Sparsity Structure

`TTM` is commonly used in popular tensor decompositions, such as the Tucker decomposition, for a variety of applications, including (social network, electrical grid) data analytics, numerical simulation, machine learning, recommendation systems, personalized web search, and so on [4, 21, 42, 63].

[3]TACO retrieved from https://github.com/tensor-compiler/taco, master@c9bd10d6. Tensors generated with `taco::util::fillTensor(tensor, taco::util::FillMethod::Sparse, 0.1)`.

Table 5. SpMSpM Involving Two Sparse Matrices

Layout A\Layout B	COO	CSR	DCSR	BCSR*
COO	1.21	1.00	0.98	✗
CSR	0.99	0.99	1.00	✗
DCSR	0.97	1.00	1.00	✗
BCSR	✗	✗	✗	1.01

We show speedup over geometric means as compared to TACO. 3 indicates representations not supported by TACO, including lower triangular and warp sparse matrix storage. For the blocked representation, BCSR, we show TACO results only against BCSR, since TACO must represent BCSR as a fourth-order tensor, requiring changes to the other tensor to express as a fourth-order tensor as well. ✗ indicates computations that are not supported by TACO, while we support all layout combinations.

Table 6. TTM (Mode-1, $R = 16$) Performance Comparison with TACO

Tensor	Collection	NNZ	TACO	Generated	Generated Parallel
Social Network Analysis					
delicious-3d	FROSTT	140M	2.07*s*	2.14*s*	0.44*s*
flickr-3d	FROSTT	113M	1.12*s*	1.20*s*	0.27*s*
freebase-music	HaTen2	100M	1.26*s*	1.30*s*	0.74*s*
Pattern Recognition					
vast-2015-mc1	FROSTT	26M	0.31*s*	0.32*s*	0.19*s*
Natural Language Processing					
NELL1	FROSTT	144M	8.38*s*	9.28*s*	0.91*s*
NELL2	FROSTT	77M	0.49*s*	0.49*s*	0.04
Anomaly Detection					
1998darpa	HaTen2	28M	0.77*s*	0.87*s*	0.20*s*

NNZ is the number of nonzeros. TACO is unable to generate parallel code due to it sequentially advancing in the positions of the sparse output.

In this experiment, we use the mode-1 variant of the computation, where the third dimension of the tensor input is contracted, $C(i, j, l) = A(i, j, k) * B(k, r)$. Note that r is typically much smaller than k in low-rank decompositions, typically $r < 100$.

TTM represents a case where there is known output sparsity when tensor A and C are stored in compressed sparse fiber format: $A(i, j, :) \neq \emptyset \rightarrow C(i, j, :) \neq \emptyset$. With our layout specification, we can describe A and C using the same auxiliary index arrays and having the same sparsity structure for the leading two dimensions.

Table 6 shows the performance of our generated code against TACO. The sparse tensors, in the **compressed sparse fiber (CSF)** layout [65], are taken from real-world applications available in the **Formidable Repository of Open Sparse Tensors and Tools (FROSTT)** [64] and the HaTen2 dataset [38]. TACO and our work require pre-generated sparsity using an assemble phase. The timing of assemble is excluded from the table. The performances of our generated code and TACO are similar. However, we are a little slower by reading one more index array for the memory location of the output variable, whereas TACO uses a counter variable for the location. However, TACO cannot generate a parallel compute code due to sequentially writing to the

output sparsity structure. In contrast, we can generate efficient parallelizable implementations from sparsity sharing and are, on a geometric average, 4.27× faster.

9 RELATED WORK

Our work can be considered as an extension to the Sparse Polyhedral Framework. This work presents a layout specification that can be integrated with the polyhedral framework and a code generation algorithm that combines polyhedral scanning with code synthesis of find algorithms.

9.1 Layout Specification

There is little work targeted to general layout specification due to it having strong ties to a specific code generation strategy. Sparse libraries may expect a standard layout for the whole tensor [33, 36, 79]. Compiler-based approaches can vary the sparse implementation of specific dimensions of the tensor using a set of names to identify known formats. Bik [11], Bik and Wijshoff [12], Pugh and Shpeisman [54] define the implementation of each index dimension of a tensor using a set of names. This approach is refined with TACO [41] by Chou et al. [20].

Unlike these prior sparse tensor compilers, our work describes the spaces of the layouts separated from the space of the computation. This enables working with blocked layouts that have a 2:1 mapping from layout dimensions to coordinate dimensions and experimenting with loop orders involving dimensions from layouts and those from the computation.

9.2 Code Generation

Tensor contraction engine [6] is a pioneering work to automatically generate dense tensor contraction computations in quantum chemistry, used extensively in the NWChem software suite [74]. It can automatically determine the binary contraction order for multiple tensor contractions with minimal operation and memory cost and define and reuse intermediate contraction results.

Bik [11], Bik and Wijshoff [12] described a compiler that can transform dense loops over dense arrays into sparse loops over nonzero elements using a technique called guard encapsulation. It treats the index set of an input tensor as a whole to define guard conditions that either include or exclude the inner computation. The Bernoulli project [43–45, 50, 67] generates sparse algebra computations by modeling the iterations as DO-ANY loops and formulates the computation as query expressions. It introduces *external* fields to represent dimensions not part of the index coordinates and index hierarchy for preferred ordering of enumeration within a layout. These previous works on sparse tensor algebra are refined with TACO [41], which formalizes the dependence between indices using the iteration graph, defines merge lattices for co-iteration, and uses a set of level format and level functions to describe the computation. Pugh and Shpeisman [54] used Enumerator/Accessors to guide the choice of layouts and code generation for sparse computation.

Other frameworks leverage the inspector-executor pattern to achieve data and computation optimization targeting specific sparsity patterns in the tensor. Sparso [59] enables context-driven optimizations using input matrix properties and matrix reordering. Comet [73] implements a tensor contraction dialect in **Multi-Level IR compiler (MLIR)** infrastructure [47]. It uses a similar layout specification as TACO and implements data reordering [49] to improve spatial and temporal locality.

9.3 Polyhedral Frameworks and Sparse Polyhedral Frameworks

The polyhedral framework can be employed on sparse arrays that does not employ uninterpreted functions. Augustine et al. [7] used trace reconstruction to exploit regular patterns in the sparse matrix. Sublimation [75] turns irregular accesses into regular accesses by exploiting the injectivity of access functions and expanding the irregular loops to regular loops to cover a larger range of

Table 7. Comparing this Work with Related Works in Sparse Polyhedral Framework (SPF) and TACO

Related Works	Co-iteration			Layout Description	Nonzero ordering	Index transformations	
	∧	∨	AS*			Blocking	Subsection
SPF [53, 77, 78]	✗	✗	✗	✗	✓	Limited	✓
TACO [20, 34, 41]	✓	✓	✗	✓	Limited	Limited	✗
This work	✓	F	✓	✓	✓	✓	✓

Support of disjunctive co-iteration (∨) and other pattern in Hentry et al. [34] is left for future work (F). *Our work enables implementing a co-iteration with a combination of different algorithms: algorithm selection (AS).

values. Zhao et al. generate code for non-affine loop bounds using conditions and exits [83]. There is also prior work in the polyhedral framework that deals with while loops [10, 22]. However, these works are insufficient to express co-iterations, which have not only dynamic loop bounds but also dynamic conditions with multiple index arrays.

The Sparse Polyhedral Framework uses inspectors to analyze properties of sparse computations at runtime to create suitable sparse data structures and transform the original computation into executors that can use the new sparse data structure. Venkat et al. [77] described transformation recipes in the polyhedral framework with make-dense and compact operations to compose layout with other loop transformations. Venkat et al. [78] described an approach that leverages runtime dependence analysis and layout transformation to achieve wavefront execution of sparse computation. Mohammadi et al. [53] used index array properties to simplify runtime dependence checking and generate efficient inspectors. The polyhedral framework composes the relations provided to the SMT solver and simplifies the dependences used to derive the inspector.

None of these prior works systematically presents a specification of sparse layout in the polyhedral framework, and none of them supported co-iterations.

9.4 Comparison

Table 7 compares this work with both sparse polyhedral frameworks and TACO [41]. Compared to prior works of sparse polyhedral frameworks, we are the first to enable sparse layout description and support co-iteration generally. These advances mean that no comparison with prior works in SPF is possible. Compared to tensor algebra compilers such as TACO [41], we eliminated many special cases in the compiler design and extended the capability in both supporting layouts and implementing co-iteration. However, in this work, the support for disjunctive merge, which can be represented as sparse loop fusion, is future work.

10 CONCLUSION & FUTURE WORK

The polyhedral framework provides mathematical descriptions of loop nest computations that enable dependence testing, composing code transformation sequences, and generating code. This article similarly achieves this result for sparse tensor co-iteration by extending the polyhedral framework in two key ways: (1) We employ a relation from a sparse tensor layout to its logical coordinate space and compose this with the logical iteration space to derive the sparse iteration space; (2) we implement co-iteration by iterating the layout of one sparse tensor and looking up the indices of the other layout through synthesizing a *find* algorithm.

This work adds another dimension of interaction in automatic tensor code generation with architecture features such as **single instruction/multiple data (SIMD)** or tensor cores. Prior works on tensor blocking and reordering [7, 49] can be orthogonally combined to provide computation speedups using such features. As the find algorithms are synthesized in this work, we can also

leverage such hardware features for implementing architecture-specific finds. Leveraging vectorization for index comparisons in the find algorithm can provide critical speedups when the indices are sparse.

With hardware architectures' increased diversity in functional units, computation capability, and memory bandwidth, adapting data layout and computation to hardware requirements is crucial to achieving performance portability for sparse tensor computations. By proposing a flexible framework for layout description and code transformation, we have opened up more opportunities for the co-optimization of layout and computation. We believe this work is a critical step in automatically generating architecture-specific variants of data layouts and computation programs.

ACKNOWLEDGMENTS

We would like to thank John Jolly and Mahesh Lakshminarasimhan, PhD students at the University of Utah, for their help in collecting experiment results.

REFERENCES

[1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI'16)*. 265–283.

[2] NVIDIA Corporation & affiliates. 2021. Parallel Thread Execution ISA Version 7.5. Retrieved from https://docs.nvidia.com/cuda/parallel-thread-execution/index.html.

[3] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-guided synthesis. In *Proceedings of the Formal Methods in Computer-Aided Design Conference*. 1–8. DOI: https://doi.org/10.1109/FMCAD.2013.6679385

[4] Animashree Anandkumar, Rong Ge, Daniel Hsu, Sham M. Kakade, and Matus Telgarsky. 2014. Tensor decompositions for learning latent variable models. *J. Mach. Learn. Res.* 15, 1 (Jan. 2014), 2773–2832.

[5] Corinne Ancourt and François Irigoin. 1991. Scanning polyhedra with do loops. In *Proceedings of the 3rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP'91)*. Association for Computing Machinery, New York, NY, 39–50. DOI: https://doi.org/10.1145/109625.109631

[6] Alexander A. Auer, Gerald Baumgartner, David E. Bernholdt, Alina Bibireata, Venkatesh Choppella, Daniel Cociorva, Xiaoyang Gao, Robert Harrison, Sriram Krishnamoorthy, Sandhya Krishnan, Chi-Chung Lam, Qingda Lu, Marcel Nooijen, Russell Pitzer, J. Ramanujam, P. Sadayappan, and Alexander Sibiryakov. 2006. Automatic code generation for many-body electronic structure methods: The tensor contraction engine. *Molec. Phys.* 104, 2 (2006), 211–228.

[7] Travis Augustine, Janarthanan Sarma, Louis-Noël Pouchet, and Gabriel Rodríguez. 2019. Generating piecewise-regular code from irregular structures. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'19)*. Association for Computing Machinery, New York, NY, 625–639. DOI: https://doi.org/10.1145/3314221.3314615

[8] Brett W. Bader, Michael W. Berry, and Murray Browne. 2008. *Discussion Tracking in Enron Email Using PARAFAC*. Springer London, 147–163. DOI: https://doi.org/10.1007/978-1-84800-046-9_8

[9] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. 2019. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO'19)*. IEEE Press, 193–205.

[10] Mohamed-Walid Benabderrahmane, Louis-Noël Pouchet, Albert Cohen, and Cédric Bastoul. 2010. The polyhedral model is more widely applicable than you think. In *Compiler Construction*. Springer Berlin, 283–303.

[11] Aart J. C. Bik. 1996. *Compiler Support for Sparse Matrix Computations*. PhD Dissertation. Leiden University.

[12] Aart J. C. Bik and Harry A. G. Wijshoff. 1993. Compilation techniques for sparse matrix computations. In *Proceedings of the 7th International Conference on Supercomputing (ICS'93)*. Association for Computing Machinery, New York, NY, 416–424. DOI: https://doi.org/10.1145/165939.166023

[13] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. 2008. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language*

Design and Implementation (PLDI'08). Association for Computing Machinery, New York, NY, 101–113. DOI: https://doi.org/10.1145/1375581.1375595

[14] Aaron R. Bradley, Zohar Manna, and Henny B. Sipma. 2006. What's decidable about arrays? In *Verification, Model Checking, and Abstract Interpretation*. Springer Berlin, 427–442.

[15] Aydin Buluc and John R. Gilbert. 2008. On the representation and multiplication of hypersparse matrices. In *Proceedings of the IEEE International Symposium on Parallel and Distributed Processing*. 1–11. DOI: https://doi.org/10.1109/IPDPS.2008.4536313

[16] John Canny and Huasha Zhao. 2013. Big data analytics with small footprint: Squaring the cloud. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'13)*. Association for Computing Machinery, New York, NY, 95–103. DOI: https://doi.org/10.1145/2487575.2487677

[17] Chun Chen. 2012. Polyhedra scanning revisited. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'12)*. Association for Computing Machinery, New York, NY, 499–508. DOI: https://doi.org/10.1145/2254064.2254123

[18] Chun Chen, Jacqueline Chame, and Mary Hall. 2008. *CHiLL: A Framework for Composing High-level Loop Transformations*. Technical Report. Citeseer.

[19] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (OSDI'18)*. USENIX Association, 579–594.

[20] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format abstraction for sparse tensor algebra compilers. *Proc. ACM Program. Lang.* 2, OOPSLA (Oct. 2018). DOI: https://doi.org/10.1145/3276493

[21] A. Cichocki, N. Lee, I. V. Oseledets, A. Phan, Q. Zhao, and D. Mandic. 2016. Low-rank tensor networks for dimensionality reduction and large-scale optimization problems: Perspectives and challenges PART 1. *ArXiv E-prints* (Sept. 2016). arXiv:1609.00893

[22] J.-F. Collard. 1994. Space-time transformation of while-loops using speculative execution. In *Proceedings of the IEEE Scalable High Performance Computing Conference*. 429–436. DOI: https://doi.org/10.1109/SHPCC.1994.296675

[23] Timothy A. Davis. 2019. Algorithm 1000: SuiteSparse:GraphBLAS: Graph algorithms in the language of sparse linear algebra. *ACM Trans. Math. Softw.* 45, 4 (Dec. 2019). DOI: https://doi.org/10.1145/3322125

[24] Timothy A. Davis and Yifan Hu. 2011. The University of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1 (Dec. 2011). DOI: https://doi.org/10.1145/2049662.2049663

[25] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems: Theory and Practice of Software (TACAS'08/ETAPS'08)*. Springer-Verlag, Berlin, 337–340.

[26] A. Einstein. 1916. Die grundlage der allgemeinen relativitÄďtstheorie. *Annalen Der Physik* 354, 7 (1916), 769–822.

[27] Paul Feautrier. 1991. Dataflow analysis of array and scalar references. *Int. J. Parallel Program.* 20, 1 (1991), 23–53.

[28] Paul Feautrier. 1992. Some efficient solutions to the affine scheduling problem. I. One-dimensional time. *Int. J. Parallel Program.* 21, 5 (1992), 313–347.

[29] Paul Feautrier. 1992. Some efficient solutions to the affine scheduling problem. II. multidimensional time. *Int. J. Parallel Program.* 21, 6 (1992), 389–420.

[30] Sylvain Girbal, Nicolas Vasilache, Cédric Bastoul, Albert Cohen, David Parello, Marc Sigler, and Olivier Temam. 2006. Semi-automatic composition of loop transformations for deep parallelism and memory hierarchies. *Int. J. Parallel Program.* 34 (06 2006), 261–317. DOI: https://doi.org/10.1007/s10766-006-0012-3

[31] M. Griebl, C. Lengauer, and S. Wetzel. 1998. Code generation in the polytope model. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*. 106–111.

[32] Tobias Grosser, Sven Verdoolaege, and Albert Cohen. 2015. Polyhedral AST generation is more than scanning polyhedra. *ACM Trans. Program. Lang. Syst.* 37, 4 (July 2015). DOI: https://doi.org/10.1145/2743016

[33] Gaël Guennebaud, Benoît Jacob, et al. 2010. Eigen v3. Retrieved from http://eigen.tuxfamily.org.

[34] Rawn Hentry, Olivia Hsu, Rohan Yadav, Stephen Chou, Kunle Olukotun, Saman Amarasinghe, and Fredrik Kjolstad. 2021. Compilation of sparse array programming models. *Proc. ACM Program. Lang* 5, OOPSLA (Oct. 2021).

[35] Eun-Jin Im and Katherine A. Yelick. 2001. Optimizing sparse matrix computations for register reuse in SPARSITY. In *Proceedings of the International Conference on Computational Sciences-Part I (ICCS'01)*. Springer-Verlag, Berlin, 127–136.

[36] Intel Corporation. 2022. Intel oneAPI Math Kernel Library. Retrieved from https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html.

[37] F. Irigoin and R. Triolet. 1988. Supernode partitioning. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'88)*. Association for Computing Machinery, New York, NY, 319–329. DOI: https://doi.org/10.1145/73560.73588

[38] Inah Jeon, Evangelos E. Papalexakis, U. Kang, and Christos Faloutsos. 2015. HaTen2: Billion-scale tensor decompositions. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*.

[39] Wayne Kelly. 1998. *Optimization within a Unified Transformation Framework*. PhD Dissertation. University of Maryland.

[40] Malik Khan, Protonu Basu, Gabe Rudy, Mary Hall, Chun Chen, and Jacqueline Chame. 2013. A script-based autotuning compiler system to generate high-performance CUDA code. *ACM Trans. Archit. Code Optim.* 9, 4 (Jan. 2013). DOI: https://doi.org/10.1145/2400682.2400690

[41] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The tensor algebra compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (Oct. 2017). DOI: https://doi.org/10.1145/3133901

[42] T. Kolda and B. Bader. 2009. Tensor decompositions and applications. *SIAM Rev.* 51, 3 (2009), 455–500.

[43] Vladimir Kotlyar and Keshav Pingali. 1997. Sparse code generation for imperfectly nested loops with dependences. In *Proceedings of the 11th International Conference on Supercomputing*. 188–195.

[44] Vladimir Kotlyar, Keshav Pingali, and Paul Stodghill. 1997. *Compiling Parallel Sparse Code for User-defined Data Structures*. Technical Report. Cornell University.

[45] Vladimir Kotlyar, Keshav Pingali, and Paul Stodghill. 1997. A relational approach to the compilation of sparse matrix programs. In *Proceedings of the European Conference on Parallel Processing*. Springer, 318–327.

[46] Daniel Langr and Pavel Tvrdik. 2016. Evaluation criteria for sparse matrix storage formats. *IEEE Trans. Parallel Distrib. Syst.* 27, 2 (2016), 428–440.

[47] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A Compiler Infrastructure for the End of Moore's Law. arXiv:2002.11054 [cs.PL]

[48] Vincent Lefebvre and Paul Feautrier. 1998. Automatic storage management for parallel programs. *Parallel Comput.* 24, 3-4 (May 1998), 649–671.

[49] Jiajia Li, Bora Uçar, Ümit V. Çatalyürek, Jimeng Sun, Kevin Barker, and Richard Vuduc. 2019. Efficient and effective sparse tensor reordering. In *Proceedings of the ACM International Conference on Supercomputing (ICS'19)*. Association for Computing Machinery, New York, NY, 227–237. DOI: https://doi.org/10.1145/3330345.3330366

[50] Nikolay Mateev, Keshav Pingali, Paul Stodghill, and Vladimir Kotlyar. 2000. Next-generation generic programming and its application to sparse matrix computations. In *Proceedings of the 14th International Conference on Supercomputing*) *(ICS'00)*. Association for Computing Machinery, New York, NY, 88–99.

[51] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. arXiv:2104.08378 [cs.LG]

[52] Mahdi Soltan Mohammadi, Kazem Cheshmi, Maryam Mehri Dehnavi, Anand Venkat, Tomofumi Yuki, and Michelle Mills Strout. 2019. Extending index-array properties for data dependence analysis. In *Languages and Compilers for Parallel Computing*. Springer International Publishing, Cham, 78–93.

[53] Mahdi Soltan Mohammadi, Tomofumi Yuki, Kazem Cheshmi, Eddie C. Davis, Mary Hall, Maryam Mehri Dehnavi, Payal Nandy, Catherine Olschanowsky, Anand Venkat, and Michelle Mills Strout. 2019. Sparse computation data dependence simplification for efficient compiler-generated inspectors. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'19)*. Association for Computing Machinery, New York, NY, 594–609. DOI: https://doi.org/10.1145/3314221.3314646

[54] William Pugh and Tatiana Shpeisman. 1999. SIPR: A new framework for generating efficient code for sparse matrix computations. In *Languages and Compilers for Parallel Computing*. Springer Berlin, 213–229.

[55] Fabien Quilleré and Sanjay Rajopadhye. 2000. Optimizing memory usage in the polyhedral model. *ACM Trans. Program. Lang. Syst.* 22, 5 (2000), 773–815.

[56] Fabien Quilleré, Sanjay Rajopadhye, and Doran Wilde. 2000. Generation of efficient nested loops from polyhedra. *Int. J. Parallel Program.* 28, 5 (Oct. 2000), 469–498. DOI: https://doi.org/10.1023/A:1007554627716

[57] J. Ramanujam and P. Sadayappan. 1992. Tiling of iteration spaces for multicomputers. In *Proceedings of the International Conference on Parallel Processing*. 179–186.

[58] M. M. G. Ricci and T. Levi-Civita. 1900. Méthodes de calcul différentiel absolu et leurs applications. *Math. Ann.* 54, 1 (Mar. 1900), 125–201. DOI: https://doi.org/10.1007/BF01454201

[59] H. Rong, J. Park, L. Xiang, T. A. Anderson, and M. Smelyanskiy. 2016. Sparso: Context-driven optimizations of sparse linear algebra. In *Proceedings of the International Conference on Parallel Architecture and Compilation Techniques (PACT'16)*. 247–259.

[60] Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (2nd ed.). Society for Industrial and Applied Mathematics. DOI: https://doi.org/10.1137/1.9780898718003

[61] Ryan Senanayake, Changwan Hong, Ziheng Wang, Amalee Wilson, Stephen Chou, Shoaib Kamil, Saman Amarasinghe, and Fredrik Kjolstad. 2020. A sparse iteration space transformation framework for sparse tensor algebra. In *Proceedings of the Conference on Object-oriented Programming, Systems, Languages, and Applications*.

[62] Tina Shen and David Wonnacott. 1998. Automatic memory remapping for time skewing. In *Proceedings of the Mid-Atlantic Student Workshop on Programming Languages and Systems (MASPLAS)*.

[63] N. D. Sidiropoulos, L. De Lathauwer, X. Fu, K. Huang, E. E. Papalexakis, and C. Faloutsos. 2017. Tensor decomposition for signal processing and machine learning. *IEEE Trans. Sig. Process.* 65, 13 (July 2017), 3551–3582.

[64] Shaden Smith, Jee W. Choi, Jiajia Li, Richard Vuduc, Jongsoo Park, Xing Liu, and George Karypis. 2017. *FROSTT: The Formidable Repository of Open Sparse Tensors and Tools*. Retrieved from http://frostt.io/.

[65] Shaden Smith, Niranjay Ravindran, Nicholas D. Sidiropoulos, and George Karypis. 2015. SPLATT: Efficient and parallel sparse tensor-matrix multiplication. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*. 61–70.

[66] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*) *(ASPLOS XII)*. Association for Computing Machinery, New York, NY, 404–415. DOI: https://doi.org/10.1145/1168857.1168907

[67] Paul Vinson Stodghill. 1997. *A Relational Approach to the Automatic Generation of Sequential Sparse Matrix Codes*. Ph. D. Dissertation. Cornell University.

[68] Michelle Mills Strout, Larry Carter, and Jeanne Ferrante. 2003. Compile-time composition of run-time data and iteration reorderings. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'68)*. ACM, New York, NY.

[69] Michelle Mills Strout, Larry Carter, Jeanne Ferrante, and Beth Simon. 1998. Schedule-independent storage mapping for loops. In *Proceedings of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems*. 24–33.

[70] M. M. Strout, M. Hall, and C. Olschanowsky. 2018. The sparse polyhedral framework: Composing compiler-generated inspector-executor code. *Proc. IEEE* 106, 11 (Nov., 2018), 1921–1934.

[71] Michelle Mills Strout, Alan LaMielle, Larry Carter, Jeanne Ferrante, Barbara Kreaseck, and Catherine Olschanowsky. 2016. An approach for code generation in the sparse polyhedral framework. *Parallel Comput.* 53, C (Apr. 2016), 32–57.

[72] William Thies, Frédéric Vivien, and Saman Amarasinghe. 2007. A step towards unifying schedule and storage optimization. *ACM Trans. Program. Lang. Syst.* 29, 6 (2007), 34.

[73] Ruiqin Tian, Luanzheng Guo, Jiajia Li, Bin Ren, and Gokcen Kestor. 2021. A high performance sparse tensor algebra compiler in MLIR. In *Proceedings of the 7th Annual Workshop on the LLVM Compiler Infrastructure in HPC*.

[74] M. Valiev, E. J. Bylaska, N. Govind, K. Kowalski, T. P. Straatsma, H. J. J. Van Dam, D. Wang, J. Nieplocha, E. Apra, T. L. Windus, and W. A. de Jong. 2010. NWChem: A comprehensive and scalable open-source solution for large scale molecular simulations. *Comput. Phys. Commun.* 181, 9 (2010), 1477–1489. DOI: https://doi.org/10.1016/j.cpc.2010.04.018

[75] Harmen L. A. van der Spek and Harry A. G. Wijshoff. 2011. Sublimation: Expanding data structures to enable data instance specific optimizations. In *Languages and Compilers for Parallel Computing*. Springer Berlin, 106–120.

[76] Nicolas Vasilache, Cedric Bastoul, Albert Cohen, and Sylvain Girbal. 2006. Violated dependence analysis. In *Proceedings of the 20th Annual International Conference on Supercomputing (ICS'06)*. Association for Computing Machinery, New York, NY, 335–344. DOI: https://doi.org/10.1145/1183401.1183448

[77] Anand Venkat, Mary Hall, and Michelle Strout. 2015. Loop and data transformations for sparse matrix code. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'15)*. Association for Computing Machinery, New York, NY, 521–532.

[78] Anand Venkat, Mahdi Soltan Mohammadi, Jongsoo Park, Hongbo Rong, Rajkishore Barik, Michelle Mills Strout, and Mary Hall. 2016. Automating wavefront parallelization for sparse matrix computations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'16)*. IEEE Press.

[79] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors.2020. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nat. Meth.* 17 (2020), 261–272. DOI: https://doi.org/10.1038/s41592-019-0686-2

[80] Minjie Wang, Lingfan Yu, Da Zheng, Quan Gan, Yu Gai, Zihao Ye, Mufei Li, Jinjing Zhou, Qi Huang, Chao Ma, Ziyue Huang, Qipeng Guo, Hao Zhang, Haibin Lin, Junbo Zhao, Jinyang Li, Alexander J. Smola, and Zheng Zhang. 2019. Deep graph library: Towards efficient and scalable deep learning on graphs. *CoRR* abs/1909.01315 (2019).

[81] Michael E. Wolf and Monica S. Lam. 1991. A data locality optimizing algorithm. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'91)*. Association for Computing Machinery, New York, NY, 30–44. DOI: https://doi.org/10.1145/113445.113449

[82] M. Wolfe. 1989. More iteration space tiling. In *Proceedings of the ACM/IEEE Conference on Supercomputing*. 655–664. DOI: https://doi.org/10.1145/76263.76337

[83] Jie Zhao, Michael Kruse, and Albert Cohen. 2018. A polyhedral compilation framework for loops with dynamic data-dependent bounds. In *Proceedings of the 27th International Conference on Compiler Construction (CC'18)*. Association for Computing Machinery, New York, NY, 14–24. DOI: https://doi.org/10.1145/3178372.3179509

Received 1 June 2022; revised 25 August 2022; accepted 30 September 2022